7

Лекция 23–24 марта

Регулярные выражения

Константин Чухарев

Дискретная математика — Весна 2027

Регулярные выражения

> *Аналитическая машина ткёт алгебраические узоры так же, как жаккардовый станок ткёт цветы и листья.*
>
> ***— Ада Лавлейс***

Два описания языка

Автомат работает как *распознаватель*. Он читает слово и отвечает, принадлежит ли оно языку.

Нужна и *запись*, то есть короткая формула, из которой язык восстанавливается без машины.

Стивен Клини ввёл регулярные выражения в 1951 году как линейную запись языка, заданного автоматом. Звезда сжимает цикл диаграммы до одного знака, поэтому язык машины укладывается в строку из трёх операций.

Автомат исполняет, выражение записывает.

Теорема Клини покажет, что это две стороны одного класса языков.

Регулярное выражение

Запись строится рекурсивно из трёх атомов и трёх операций.

Определение 1

Регулярное выражение над Σ строится по следующим правилам.

- $\varnothing$ — пустой язык
- ε — язык $\{\varepsilon\}$ из одного пустого слова
- $a \in \Sigma$ — язык $\{a\}$
- $R_1 \mid R_2$ — объединение языков
- $R_1 R_2$ — конкатенация
- R^* — звезда, ноль и более повторений

Из правил выходит дерево разбора. Листья — атомы, внутренние узлы — операции, и язык выражения вычисляется по этому дереву.

Синтаксис, приоритет и скобки

Приоритет снимает двусмысленность записи. Каждая строка разбирается в одно дерево.

Запись	Читается как	Язык
ab^*	$a(b^*)$	слово a, затем сколько угодно b
$ab \mid c$	$(ab) \mid c$	слово ab или слово c
$a \mid bc$	$a \mid (bc)$	слово a или слово bc

Похожие записи ab^* и $(ab)^*$ задают разные языки.

Слово abb лежит в первом и не лежит во втором. Во втором повторяется пара ab, а не буква b.

Дерево разбора для $(a \mid b)^*abb$

Пример

Соберём дерево разбора для $(a \mid b)^*abb$.

1. Звезда связывает сильнее всех, и $(a \mid b)^*$ становится узлом с выбором $a \mid b$ внутри.
2. Конкатенация собирает четыре множителя и группирует их слева, в $((a \mid b)^*ab)b$.

```
concat
  concat
    concat
      star(a | b)
      a
    b
  b
```

Узлов в дереве линейно от длины записи. Конструкция Томпсона строит по автомату на каждый узел, поэтому состояний у неё $O(n)$.

Примеры выражений

Пример

Разберём выражения над алфавитом $\{0, 1\}$.

- $(0 \mid 1)^*$ — все двоичные слова, включая пустое, например 0, 110 и ε.
- 01^*0 — слова с нулями по краям и единицами в середине, например 00, 010 и 0110.
- $(0 \mid 1)^*01$ — слова с хвостом 01, например 01, 001 и 1101.
- $(0 \mid 1)(0 \mid 1)^*$ — все непустые слова, например 0, 10 и 011.

$\emptyset$, ε и 0^* — три разных языка. В первом нет слов, во втором одно слово длины 0, в третьем слова из нулей любой длины.

Язык выражения через структурную рекурсию

Язык $L(R)$ выражения R определяется вместе с самим выражением.

Определение 2

- $L(\emptyset) = \emptyset$, $L(\varepsilon) = \{\varepsilon\}$, $L(a) = \{a\}$ для $a \in \Sigma$
- $L(R \mid S) = L(R) \cup L(S)$
- $L(RS) = \{uv \mid u \in L(R), v \in L(S)\}$
- $L(R^*) = \cup_{k \geq 0} L(R)^k$, где $L(R)^0 = \{\varepsilon\}$

Пример

Вычислим язык выражения $(a \mid b)^*abb$ по шагам.

$$L((a \mid b)^*abb) = \{a, b\}^*\{a\}\{b\}\{b\} = \{uabb \mid u \in \{a, b\}^*\},$$

то есть слова с хвостом abb. Например, aabb и abb лежат в языке, a ab и abab — нет.

Алгебра регулярных выражений

Тождество выражений — равенство двух языков, и проверяется оно как равенство множеств слов.

Тождество	Имя
$\varepsilon R = R\varepsilon = R$	нейтральность пустого слова
$\varnothing R = R\varnothing = \varnothing$	аннулирование
$(RS)T = R(ST)$	ассоциативность конкатенации
$R(S \mid T) = RS \mid RT$	дистрибутивность
$\varepsilon \mid RR^* = R^*$	развёртка звезды
$(R^*)^* = R^*$	идемпотентность звезды

Доказательство идёт взаимным включением. Слово раскладывается по построению и собирается обратно.

Наброски доказательств тождеств

Доказательство (набросок)

Все три тождества докажем разложением слова по построению.

Ассоциативность. Слово из $(RS)T$ имеет вид $(uv)w$ при $u \in L(R)$, $v \in L(S)$, $w \in L(T)$. Скобки в конкатенации слов не меняют слово, $(uv)w = u(vw)$, поэтому оно лежит в $R(ST)$. Обратное включение — то же равенство справа налево.

Дистрибутивность. Слово из $R(S \mid T)$ равно uv, где v лежит в $L(S)$ или в $L(T)$. В первом случае оно лежит в RS, во втором — в RT, значит и в объединении. Обратно, слово из $RS \mid RT$ имеет вид uv с $v \in L(S)$ или $v \in L(T)$.

Развёртка звезды. Слева стоит пустое слово либо слово $r_1 \ldots r_k$ при $k \geq 1$, и оба лежат в R^*. Обратно, слово из R^* либо пустое, либо раскладывается как $r_1(r_2 \ldots r_k)$ с $k \geq 1$, то есть лежит в RR^*. При $R = a$ получаем $\varepsilon \mid aa^* = a^*$.

Идемпотентность звезды

Доказательство (набросок)

Докажем оба включения по определению звезды.

Включение $(R^*)^* \subseteq R^*$. Слово из $(R^*)^*$ — конкатенация кусков, каждый из которых лежит в R^*. Каждый кусок — конкатенация слов из $L(R)$, поэтому вся конкатенация снова лежит в R^*.

Обратное включение. Слово из R^* лежит в $(R^*)^*$ как одно повторение куска.

При $R = a \mid b$ оба языка совпадают с $(a \mid b)^*$, и слово b a b лежит в каждом.

Алгебра

1. Упростите $(a \mid \varepsilon)^*$ и обоснуйте упрощение определением звезды.
2. Проверьте на словах тождество $(RS)^*R = R(SR)^*$.

Похожие законы, которые ложны

Пример

Закон $(R \mid S)^* = R^* S^*$ похож на дистрибутивность и ложен.

Возьмём $R = a$ и $S = b$. Слово b a лежит в $(a \mid b)^*$, но не в $a^* b^*$, где все a идут раньше всех b.

Конкатенация некоммутативна. Слово a b лежит в языке ab и не лежит в языке ba, поэтому языки $L(ab)$ и $L(ba)$ различны.

Привычка к коммутативности сложения и умножения здесь не работает.

Теорема Клини

Класс языков автоматов обозначается AUT, класс языков выражений — REG.

Теорема 1 (Клини, 1956)

Язык регулярен тогда и только тогда, когда он представим регулярным выражением.

$$\mathrm{AUT} = \mathrm{REG}.$$

Доказательство (набросок)

Докажем двумя построениями, по одному на направление. От выражения к автомату ведёт структурная индукция, где атомы дают кирпичи из пары состояний, а операции склеивают их ε-переходами. От автомата к выражению ведёт рекуррентность Клини, где состояниям приписываются выражения для слов, ведущих из старта.

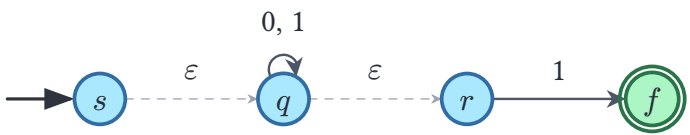
Каждое направление даёт алгоритм перевода, поэтому запись и автомат вычислимо переходят друг в друга.

От выражения к автомату

Индукция по записи строит автомат снизу вверх.

База. Атомы a, ε и $\emptyset$ — автоматы из пары состояний.

Шаг. Объединение и конкатенация склеивают автоматы ε-переходами, а звезда замыкает тело в петлю и пропускает пустое слово мимо тела.



На рисунке — автомат для $(0 \mid 1)^*1$ после упрощения. Тело звезды стало петлёй q по 0 и 1, а конкатенация — ε-переходом в r и ребром по 1 в f.

Сборка и прогон $(0 \mid 1)^*1$

Пример (Полная сборка)

Автомат собран из кирпичей. Из s в q ведёт ε-ребро входа, петля q читает 0 и 1, из q в r идёт ε-ребро выхода, а из r в f — ребро по 1. Принимающее состояние одно, это f.

Прогоним слово 01. Старт даёт ε -closure$(\{s\}) = \{s, q, r\}$. По 0 ведёт только петля q, и замыкание добавляет r, так что живым остаётся $\{q, r\}$. По 1 петля держит q, а r переходит в f, и замыкание возвращает r по ε-ребру из q. Живым остаётся множество $\{q, f, r\}$, и слово принято.

Пустое слово кончается в $\{s, q, r\} - f$ не достигнут, и звезда честно не принимает ε.

От автомата к выражению

В алгоритме Клини состояния нумеруются, и через R_{ij}^{k} обозначаются слова, переводящие из q_i в q_j путями с промежуточными состояниями не выше k.

Базой служат переходы автомата, а при $i = j$ к ним добавляется пустое слово. Шаг индукции применяет рекуррентность.

$$R_{ij}^{k} = R_{ij}^{k-1} \cup R_{ik}^{k-1} \left(R_{kk}^{k-1}\right)^{*} R_{kj}^{k-1}.$$

Путь либо не заходит в q_k, либо проходит его цикл сколько угодно раз.

Это тот же приём, что в алгоритме Флойда—Уоршелла, динамическое программирование по промежуточным вершинам. Разница в алгебре, где вместо min и сложения работают объединение, конкатенация и звезда.

Выражение выходит огромным, но строится для любого автомата и всегда конечно.

Устранение состояний

Есть и второй путь от автомата к выражению. Состояния удаляются по одному, и их работа переносится на рёбра.

Определение 3

Рёбра автомата помечаются регулярными выражениями, добавляются вход s с ε-переходом в старт и выход t с ε-переходами из принимающих состояний.

Устранение состояния q превращает каждое ребро $i \to j$ в обход через q.

$$R_{ij} \quad \cup \quad R_{iq}(R_{qq})^* R_{qj}.$$

Когда состояний не остаётся, метка единственного ребра $s \to t$ — выражение языка.

Формула повторяет рекуррентность алгоритма Клини. Путь либо обходит q, либо прокручивает его петлю, а состояния устраняются целиком.

Устранение состояний для $(a \mid b)^* ab$

Автомат для слов, заканчивающихся на ab, имеет три состояния.

Состояние	**по a**	**по b**
q_0 — старт	q_1	q_0
q_1 — хвост a	q_1	q_2
q_2 — принято	q_1	q_0

Пример

На шаге 1 устраняем q_1. Входы в него — рёбра a из q_0 и q_2, петля a, а выход — ребро b в q_2, поэтому ребро $q_0 \to q_2$ получает метку aa^*b.

Петля q_2 получает ту же метку aa^*b, а ребро $q_2 \to q_0$ по b не меняется.

В исходном автомате слово b a b приходит в q_2, а a b b — в q_0.

Шаги 2 и 3 устранения

Пример

На шаге 2 устраняем q_0. Входы — ε от входа s и b из q_2, выходы — петля b и ребро aa^*b.

$$s \to q_2 : \varepsilon b^*(aa^*b) = b^*aa^*b, \quad q_2 \to q_2 : aa^*b \cup bb^*(aa^*b).$$

На шаге 3 устраняем q_2, и метка ребра $s \to t$ равна $b^*aa^*b(aa^*b \cup bb^*aa^*b)^*$.

Петлю упрощает развёртка звезды, $aa^*b \cup bb^*aa^*b = (\varepsilon \mid bb^*)aa^*b = b^*aa^*b$.

Выражение $b^*aa^*b(b^*aa^*b)^*$ задаёт тот же язык, что $(a \mid b)^*ab$. Кусок b^*aa^*b кончается на ab, а слово, кончающееся на ab, режется на такие куски слева направо.

Слово a a b b a b распадается на куски a a b и b a b, и каждый кусок имеет вид b^*aa^*b.

Замыкания и булевы операции

Что остаётся регулярным после операций над языками?

Теорема 2 (Замкнутость относительно булевых операций)

Если L_1 и L_2 регулярны, то регулярны $L_1 \cup L_2$, $L_1 \cap L_2$, $\overline{L_1}$ и $L_1 \setminus L_2$.

Доказательство

Докажем конструкциями, по одной на операцию.

Объединение — склейка двух ε-автоматов или запись $R_1 \mid R_2$.

Путь у ДКА один, поэтому достаточно сменить принимающие состояния.

Пересечение строится прямым произведением, то есть парами (q, r) с покомпонентными переходами и принимающими парами. Разность — это пересечение с дополнением, $L_1 \setminus L_2 = L_1 \cap \overline{L_2}$.

Дополнение через смену принимающих состояний

Пример

У ДКА для $(a \mid b)^*ab$ принимающее состояние одно, это q_2, а остальные два состояния не принимают. Поменяем роли, и принимающими станут q_0 и q_1. Автомат распознаёт все слова, которые *не* заканчиваются на ab.

Слово a b ушло из языка, слова a b a и ε появились. Путь у ДКА один, поэтому для каждого слова ответ перевернулся.

Для НКА тот же приём ломается. Путей много, и слово с неуспешным путём может иметь и принимающий.

Пересечение через прямое произведение

Пример

L_1 — слова, заканчивающиеся на 1, с ДКА из состояний p_0 и p_1 и принимающим p_1.
L_2 — слова чётной длины, с ДКА из состояний e_0 и e_1 и принимающим e_0.
Произведение живёт в парах (p, e), читает символ покомпонентно и принимает пары (p_1, e_0), то есть язык слов, которые заканчиваются на 1 и имеют чётную длину.

Слово 0 1 идёт $(p_0, e_0) \to (p_0, e_1) \to (p_1, e_0)$ и принимается, слово 1 0 1 приходит в (p_1, e_1) и отвергается.

Среди трёх операций выражений нет знака пересечения. Замкнутость класса видна только на автоматах.

Производные выражений

Совпадение с текстом можно проверять по одной букве, дифференцируя запись.

Определение 4

Производная выражения R по символу a — выражение $D_a(R)$ с языком $\{w \mid aw \in L(R)\}$.

- $D_a(\emptyset) = D_a(\varepsilon) = \emptyset$, $D_a(a) = \varepsilon$, $D_a(b) = \emptyset$ при $b \neq a$
- $D_a(R \mid S) = D_a(R) \mid D_a(S)$ и $D_a(R^*) = D_a(R)R^*$
- $D_a(RS) = D_a(R)S \mid \nu(R)D_a(S)$, где $\nu(R) = \varepsilon$ при $\varepsilon \in L(R)$ и $\emptyset$ иначе

Теорема 3 (Бржозовски, 1964)

Слово w лежит в $L(R)$ тогда и только тогда, когда $\varepsilon \in L(D_w(R))$, где D_w — производные по буквам слова. Различных производных у выражения конечное число.

Пример производных

Пример

Возьмём выражение $R = a^*b$ и текст a a b.

Буква a производную не меняет, ведь $D_a(R) = D_a(a^*)b \mid \nu(a^*)D_a(b) = a^*b$.

Буква b даёт $D_b(a^*b) = \varnothing \mid \varepsilon = \varepsilon$. Язык производной содержит пустое слово, поэтому текст принят.

Пример

На тексте a b a после a b производная равна ε, а $D_a(\varepsilon) = \varnothing$, поэтому текст отвергнут.

Различных производных у a^*b ровно три, а именно a^*b, ε и $\varnothing$. Это три состояния ДКА для того же языка, и они пересчитываются прямо на записи.

Регулярные выражения вокруг нас

Пример

Идентификатор языка программирования записывается как `[a-zA-Z_][a-zA-Z0-9_]*`, то есть буква или подчёркивание и затем любые символы имени.

Тот же язык записывается как (буква | _)(буква | цифра | _)*.

- Валидация форм, то есть телефон, почтовый индекс и адрес почты.
- Поиск и замена в редакторах и утилитах командной строки.
- Подсветка синтаксиса и генераторы лексеров.

Всё это — теорема Клини в действии. Запись компилируется в автомат, а автомат исполняет распознавание.

От grep до лексера

Кен Томпсон скомпилировал регулярное выражение в автомат для QED в 1968 году.

Тот же приём переехал в `ed`, затем в `grep`.

Лексер — первый компонент каждого компилятора. Токены задаются регулярными выражениями, компилируются в ДКА и распознаются за $O(1)$ на символ.

Регулярные языки заканчиваются на лексере. Вложенные скобки и грамматика языка требуют более сильных моделей.

Библиотеки с возвратом перебирают пути автомата по одному, и на неудачных парах выражение–текст время взрывается экспоненциально.

В теории выражение компилируется в ДКА, и распознавание линейно по тексту.

POSIX против возврата

Запись одна, движков два семейства, и совпадение они выбирают по-разному.

Свойство	Автоматные	С возвратом
Стратегия	симуляция НКА множеством	обход дерева в глубину
Худший случай	линейное время	экспоненциальное время
Правило выбора	наибольшее (POSIX) либо первое (Perl)	по порядку альтернатив
Обратные ссылки	нет	есть, и язык уже нерегулярен

Пример

Шаблон `a | ab` на слове `ab` даёт в первой позиции два совпадения, `a` и `ab`.

POSIX-движки (`grep -E`, `awk`) вернут `ab`, то есть наибольшее совпадение.

PCRE, Python `re`, RE2 и Rust `regex` вернут `a`, то есть первую альтернативу.

Катастрофический возврат

Выражение $(aa^*)^*b$ задаёт цепочку из a с одной b в конце — его ДКА имеет три состояния.

Пример

Для k повторений внешней звезды выходит $\binom{n-1}{k-1}$ разбиений цепочки из n букв.

Сумма по всем k равна 2^{n-1}, так что тридцать букв дают $2^{29} \approx 5.4 \times 10^8$ попыток.

```
import re

pattern = re.compile(r"(a+)+b")
pattern.match("a" * 30)   # 2^29 разбиений --- минуты счёта
```

Перебор идёт по одному пути, и все пути несут одно и то же, «прочитаны a, ждём b».

В июле 2019 года такой возврат в WAF-фильтре Cloudflare отключал сайты на полчаса.

Нерегулярные языки

> *В одну и ту же реку нельзя войти дважды.*
>
> ***— Гераклит***

Конечная память имеет границу

Автомат не умеет считать неограниченно.

Пример

В языке $L = \{0^n 1^n \mid n \geq 0\}$ нулей и единиц поровну, и все нули идут раньше единиц.

Для распознавания нужно помнить разность «нули минус единицы» — а она ничем не ограничена.

Утверждение «не регулярен» говорит о всех автоматах сразу. Ни один ДКА не распознаёт язык.

Для такого утверждения нужен инструмент — свойство, которым обладает *каждый* регулярный язык.

Цикл в диаграмме переходов

Пусть ДКА имеет p состояний и читает слово длины p.

Пройдено $p + 1$ позиций при p состояниях, поэтому по принципу Дирихле какое-то состояние повторилось.

Пример

Автомат с тремя состояниями читает 0001.

Пять позиций пути против трёх состояний, поэтому где-то автомат вернулся в то же место. Дальше он идёт по той же колее, сколько бы раз ни прошёл цикл.

Повтор состояния — цикл, который можно пройти ноль, один или много раз.

Автомат разницы не заметит, ведь конец пути тот же.

Лемма о накачке

Теорема 4 (Свойство накачки)

Для регулярного L найдётся $p \geq 1$, при котором каждое слово $w \in L$ длины $|w| \geq p$ имеет вид $w = xyz$ с условиями

$$|y| > 0, \quad |xy| \leq p, \quad xy^i z \in L \text{ для всех } i \geq 0.$$

Доказательство

Докажем от принципа Дирихле.

Пусть ДКА для L имеет p состояний, а r_k — состояние после k символов слова w. Среди $p+1$ состояний $r_0, r_1, \ldots, r_p$ найдутся $i < j$ с $r_i = r_j = s$. По этим номерам слово делится на x, y, z с $|y| > 0$ и $|xy| \leq p$. Чтение y возвращает автомат в s, поэтому слово $xy^k z$ принимается вместе с w при любом $k \geq 0$.

Накачка работает и для регулярных языков

Пример

У автомата для слов, заканчивающихся на 01, всего $p = 3$ состояния, а слово 0001 ведёт его по состояниям q_0, q_1, q_1, q_1, q_2.

Состояние q_1 повторилось, и разбиение готово. Возьмём $x = 0$, $y = 0$, $z = 01$. Условия $|y| > 0$ и $|xy| \leq 3$ выполнены, а накачанные слова 001, 0001, 00001 — все кончаются на 01.

Лемма описывает свойство *каждого* регулярного языка. У каждого длинного слова такого языка найдётся накачиваемое разбиение.

Значит, если для языка разбиения нет ни при каком p, язык не регулярен.

Опровержение для языка $0^n 1^n$

Доказательство (от противного)

Предположим, что $L = \{0^n 1^n\}$ регулярен, и пусть p — его длина накачки.

Возьмём слово $w = 0^p 1^p \in L$. По лемме $w = xyz$ при $|xy| \leq p$ и $|y| > 0$. Префикс xy лежит в первых p символах, а там только нули, поэтому $y = 0^k$ при $k > 0$.

Накачанное слово $xy^2z = 0^{p+k}1^p$ обязано лежать в L, но нулей в нём больше, чем единиц. Это противоречие.

Следовательно, L не регулярен.

Схема опровержения одна. Берём слово подлиннее p, накачиваем подслово в префиксе и выходим из языка.

Выбор слова — единственное место, где нужна изобретательность.

Другие нерегулярные языки

- Палиндромы над $\{a, b\}$. Возьмём $w = a^p b a^p$. Разрез попадает в первые p символов, $y = a^k$, а слово $a^{p+k} b a^p$ палиндромом уже не является.
- a^{2^n}. Накачка $y = a^k$ при $0 < k \leq p$ даёт длину $2^p + k$, строго между 2^p и 2^{p+1}.
- $a^n b^n c^n$. Накачка в первых p символах портит блок a, а остальные блоки стоят на месте.

Во всех трёх случаях схема одна. Берём длинное слово, накачиваем подслово в префиксе и выходим из языка.

Меняется только подсчёт длин.

Границы леммы

Лемма о накачке даёт необходимое условие регулярности, но не достаточное. Опровергнуть регулярность леммой можно, а доказать нельзя.

Существуют нерегулярные языки, которые лемме удовлетворяют, и на них опровержение не выходит.

Инструмент, работающий в обе стороны и строящий минимальный автомат, даёт теорема Майхилла–Нерода.

Итоги регулярных выражений

Выражение и автомат — две записи одного класса.

Замыкания держат операции внутри класса, накачка отмечает его границу.

- Регулярное выражение — запись языка из атомов и трёх операций $|$, конкатенации и звезды.
- Теорема Клини даёт $\mathrm{AUT} = \mathrm{REG}$, то есть два описания одного класса.
- Туда и обратно ведут конструкция Томпсона и алгоритм Клини с устранением состояний.
- Класс замкнут, дополнение строят сменой принимающих ДКА, пересечение прямым произведением.
- Производная Бржозовски проверяет слово по буквам и даёт автомат прямо из записи.
- Накачка ловит границу класса, ведь длинное слово регулярного языка содержит накачиваемый цикл, а 0^n1^n нарушает условие леммы.

Вопросы по регулярным выражениям

выражения, автоматы и накачка

1. Запишите выражение для слов над $\{0, 1\}$ с подстрокой 01.
2. Чем различны языки $\emptyset$ и ε и что содержит $\emptyset^*$?
3. Что в алгоритме Клини играет роль промежуточных вершин из Флойда—Уоршелла?
4. Накачкой докажите нерегулярность языка $a^n b^n c^n$.
5. Чем устранение состояния отличается от шага рекуррентности алгоритма Клини?
6. Вычислите производные ab^*a по буквам a b a. Принимается ли слово?
7. Почему дополнение строят сменой принимающих состояний ДКА, а не НКА?

Что почитать?

Задача	Где читать
Понять regex	m23, раздел «Регулярные выражения»
Замкнуть операции	m23, раздел «Свойства замкнутости»
Доказать нерегулярность	m23, раздел «Лемма о накачке»
Разобрать движки	m23, раздел «Регулярные выражения на практике: ДКА против backtracking»
Собрать лексер	m23, раздел «Лексер: регулярные выражения в компиляторе»